Information about the assembler built for demonstration 2.

In demonstration two I am including an additional `/extras` directory in the submission folder. The folder contains two *Python* documents that describe ***an assembler built for the Instruction Set defined by the Processor Architecture developed in this lab***. This file is a `README.pdf` to explain the *Application Programming Interface* (API), and where the source code can be found. For completion, **this file also acts as a summary of all the 'Added Value' assessment criteria attempted as a part of this assignment.**

Note the file structure of the `/week8` directory for reference.

```
/week8
├── TopLevel.bit
├── constraint.xdc
├── README.pdf
├── extras
│   ├── assembler.py
│   └── main.py
├── program
│   ├── Complete_RAM_demo.mem
│   └── Complete_ROM_demo.mem
├── sim
│   ├── colour_logic_test.v
│   ├── frame_buffer_test.v
│   ├── generic_counter_test.v
│   ├── pattern_state_machine_test.v
│   ├── processor_test.v
│   ├── ram_test.v
│   ├── rom_test.v
│   ├── timer_test.v
│   ├── top_stim.v
│   ├── vga_bus_test.v
│   ├── vga_clock_test.v
│   ├── vga_controller_test.v
│   └── vga_sig_gen_test.v
└── verilog
    ├── ALU.v
    ├── ClockDivider.v
    ├── ColourLogic.v
    ├── Complete_RAM_demo.mem
    ├── Complete_ROM_demo.mem
    ├── FrameBuffer.v
    ├── GenericCounter.v
    ├── PatternStateMachine.v
    ├── Processor.v
    ├── RAM.v
    ├── ROM.v
    ├── Timer.v
    ├── Top.v
    ├── VGABusInterface.v
    ├── VGADisplayController.v
    ├── VGAInterface.v
    └── VGASignalGen.v
```

Assembler

The typical approach to this problem is to treat the Instruction Set for this processor as a *domain specific language*, and then implement a lexer and parser for the language based on some developed grammar. Additionally to implement labels, a two pass approach would be needed to retroactively replace label uses with the correct instruction memory address.

Issues with this approach:

1. Implementing a parser involves complicated coding with, perhaps regular expressions, or creating abstract syntax trees and employing other annoyingly difficult algorithms/data structures that would entirely overshoot the demonstration deadline.
2. Assuming the above limitation is overcome, there is still the issue that incorrect syntaxes could be written/assembled. The purpose of an assembler is to avoid such overheads and speed up code iteration however, so this remains an issue still.
3. Additionally the lexer would have to be used with a *Language Server Protocol* (LSP) to be able to use a code editor to it's full capacity. **This is not a necessity but as the approach I have come up with has effectively solved this as well as the above, it is an important limitation to me now.**

Proposed (and implemented) approach:

Instead of implementing a full parser I've created an `Assmebler` class which is a *Python* `class` that provides individual functions for each instruction in the ISA. These instructions follow a format maintained by the `Instruction` class. The `Mnemonics` class hardcodes the peripheral memory location to **further reduce chances of mistakes.** The main speed-up in coding came from the implementation of the `here()` function which is able to add labels without needing two passes. No need to count lines in hexadecimal! This implementation is basically an ***embedded Domain Specific Language.*** The assembler is in the `assembler.py` file.

Main advantages:

1. **No parser implementation needed** — Python handles all the syntax, so no text processing, no tokeniser, no grammar definition is needed.
2. **No two-pass needed** — `here()` lets you compute forward addresses using ordinary Python arithmetic before emitting the instruction.
3. Python as a macro system — **you can use loops, functions, conditionals to generate repetitive instruction sequences**, **which a traditional assembler would need a separate macro language for**.
4. **Type checking and autocomplete — your IDE LSP knows what methods exist on** `Assembler`. This greatly speeds up the coding, especially combined with the `here()` API.
5. **Automatically add annotations of memory locations in ROM to comments in** `.txt` **output file.** Makes it easier to understand and navigate the code in addition to user comments.

Limitations:

This works because the ISA is extremely small. When done professionally it should always be done by the typical method and using professional compiler design practices and standards.

Application Programming Interface

`Assembler` This class provides the functions encoding opcodes (and operands if needed), adding them to the instruction cache. It also provides the interface for dumping it into a `.txt` file that can be used to copy-paste into the file *Vivado* is using.

`Instruction` This is a *Python* `dataclass` that provides the structure for instructions to maintain consistent format.

`Menomnics` This is a *Python* `dataclass` that provides defined mnemonics for memory acces. **This will be expanded further to add more functionality for RAM 'compilation'**.

Assembler:

Method	Arguments	Description	Bytes
Memory			
`load`	`reg, memloc`	Load `[memloc]` into `A` or `B`	2
`store`	`reg, memloc`	Write `A` or `B` to `[memloc]`	2
ALU — Arithmetic			
`add`	`dest=A`	`dest = A + B`	1
`sub`	`dest=A`	`dest = A - B`	1
`mul`	`dest=A`	`dest = A * B`	1
`shift_left`	—	`A = A « 1`	1
`shift_right`	—	`A = A » 1`	1
`inc`	`reg`	`reg = reg + 1`	1
`dec`	`reg`	`reg = reg - 1`	1
ALU — Comparison			
`eq`	`dest=A`	`dest = 1` if `A == B`, else `0`	1
`gt`	`dest=A`	`dest = 1` if `A > B`, else `0`	1
`lt`	`dest=A`	`dest = 1` if `A < B`, else `0`	1
Branches & Jumps			
`breq`	`addr`	Goto `addr` if `A == B`	2
`bgtq`	`addr`	Goto `addr` if `A > B`	2
`bltq`	`addr`	Goto `addr` if `A < B`	2
`goto`	`addr`	Unconditional jump to `addr`	2
`goto_idle`	—	Suspend; wait for next interrupt	1
Functions			
`call`	`addr`	Save return address; jump to `addr`	2
`ret`	—	Return from function call	1
Dereference			
`deref`	`reg`	`A <- [A]` or `B <- [B]`	1
Utility			
`here`	—	Returns current byte address as a Python `int`	—
`pad_to`	`addr`	Fill with `0x08` (GOTO_IDLE) bytes up to `addr`	var
`db`	`value`	Place a single raw byte (e.g. interrupt vector entries)	1
`section_comment`	`comment`	Emit `// ==== LABEL (0xAB) ====` with current address	—
`inline_comment`	`comment`	Emit a bare comment line	—
`create_file`	`name, version`	Serialise all instructions to `asmfiles/<name>-v<version>.txt`	—

Mnemonics:

Mnemonic field	Address	Range	Description
Data Memory			

—	0x00	0x00–0x7F	RAM (128 bytes, read/write)
Peripherals			
ir_base	0x90	0x90	IR Transmitter (bonus)
mouse_status	0xA0	0xA0–0xA2	Mouse status byte
mouse_x	0xA1	—	Mouse X direction byte
mouse_y	0xA2	—	Mouse Y direction byte
vga_write	0xB0	0xB0–0xB2	VGA framebuffer write trigger (Y≪1|pixel)
vga_x	0xB1	—	VGA X pixel address
vga_config	0xB2	—	VGA colour config (Config_Colours[15:0])
leds_base	0xC0	0xC0	LED output register
sevenseg_base	0xD0	0xD0–0xD1	Seven segment display
timer_value	0xF0	0xF0–0xF3	Current timer value (read)
timer_rate	0xF1	—	Interrupt interval in ms (default 100)
timer_reset	0xF2	—	Write to reset timer to zero
timer_enable	0xF3	—	Write 1/0 to enable/disable timer interrupt
Interrupt Vectors (ROM)			
timer_vector	0xFE	—	Address of timer interrupt handler
mouse_vector	0xFF	—	Address of mouse interrupt handler

Using the API

The code currently implemented on the ROM was made using this API (with some last minute adjustments directly in the hex). This example is included in the `main.py` file in the submission. The API is recommended to use with populating the `__main__` namespace with the following, prior to using the `Assembler` class function. This enables working even faster.

```
M   = Mnemonics()
asm = Assembler()
A, B = Assembler.A, Assembler.B
```

Here is a demonstration code block:

```
asm.section_comment("INIT - load constants, zero state")
asm.load(A, 0x05, "const 0")
asm.store(A, M.timer_enable, "disable timer interrupt")
asm.store(A, 0x00, "X = 0")
asm.store(A, 0x01, "Y = 0")
asm.load(A, 0x06, "const 1")
asm.store(A, 0x03, "x_even = 1")
asm.store(A, 0x04, "y_even = 1")
```

Other Additional Contributions

Given the recommended `verilog` implementations of the various components of the system, some changes were made to facilitate hardware optimisation and correct certain aspects of the hardware definition itself. These are described in the list below.

`RAM.v` Originally the RAM addressing logic used, what the comment called 'brute force' addressing, which

basically checks lower and higher limits of the RAM address allocation versus the current bus address. This involves implementing 3 separate operators in the hardware and can be optimised.

```
...
// Brute-force RAM address decoding. Think of a simpler way...
if((BUS_ADDR >= RAMBaseAddr) & (BUS_ADDR < RAMBaseAddr + 128)) begin
...
```

This has been changed to the following code block. This is based on the observation that for ALL RAM adresses, and only for all the RAM addresses, the *Most Significant Bit* (MSB) is 0. So we take a NOT and then do ram operations. This reduced the number of operators implemented to 1.

```
...
// The brute force adressing here has been replaced
if(!BUS_ADDR[7]) begin
...
```

`Timer.v` This module was supposed to implement a recurring timer interrupt with a 100ms period. This, when used in a 10x loop to create a 1s delay before switching the colour on the VGA, resulted in a 0.5s delay instead. Looking at the clock decimation, the counter max was 49999 and not 99999 resulting in a 50ms period and not a 100ms period. This, has been changed with additional optimisations.

Change	**Location**	**What changed**
Decimation Correction	`DownCounter`	`49999` → `99999`
Timer reset now write-only	Timer always block	Split `RESET | addr` into separate `if/else if`, added `BUS_WE` guard
`TransmitTimerValue` reset added	`TransmitTimerValue` block	Added `if(RESET)` branch initialising to zero.
Bus contention on write fixed	`TransmitTimerValue` block	Added `&& !BUS_WE` to prevent driving bus during processor writes
`&` → `&&`	`InterruptRate`, `InterruptEnable`	Logical AND instead of bitwise AND
`else Timer <= Timer` removed	Timer always block	Redundant self-assignment removed